Audio for Contact-Rich Manipulation

Paolo Andoni
Supervised by Rickmer Krohn
September 2026

Abstract

Cameras often miss what happens at the moment of contact: when a gripper wipes a table or pushes a peg into a hole, the contact itself is hidden by the gripper and the part. The sound of the contact can fill this gap. This project lays the groundwork for adding audio as a sensor to a robot that already uses vision and its own joint state.

Based on a review of the literature on audio in robot manipulation, a ROS 2 pipeline was built. It records audio, converts it into a log-mel spectrogram, encodes it with a neural network, and publishes the resulting features so that a policy can use them together with camera images and robot state. After recording, each piece of audio is matched with the robot's position at the same time, and the robot's actions are computed from its movement. This matching was tested with a simulated robot. First results are encouraging. In hand recordings made with an air microphone and substitute objects, a classifier using pretrained Audio Spectrogram Transformer features identified the correct sound category in 15 of 18 clips. These recordings support the planned cleaning and peg-insertion study. In an early test on a published contact-microphone dataset, audio features gave lower motion-prediction error than a constant baseline.

The next step is to mount a contact microphone on the gripper of the Franka arm, measure the robot's own noise, record teleoperated demonstrations, and compare policies trained with and without audio by their task success on the real robot.

Contents

1 Literature Review **2**
- 1.1 Sound as a sensor for manipulation . . . 2
- 1.2 Combining sound with vision and touch . . . 2
- 1.3 Learning manipulation from demonstrations . . . 2
- 1.4 Audio representations . . . 2
- 1.5 General points and relation to this project . . . 2

2 Motivation and Research Questions **3**

3 Project Scope **3**

4 From Audio Waveform to a ROS 2 Pipeline **3**
- 4.1 The audio representation . . . 3
- 4.2 Two nodes . . . 4
- 4.3 Where robot state enters . . . 4
- 4.4 Observations and actions for a policy . . . 5

5 Real-Robot Evaluation Plan **5**
- 5.1 Choosing a task . . . 5
- 5.2 How the robot uses the audio . . . 5
- 5.3 The lab session . . . 6
- 5.4 Plan for the next session . . . 6

6 Evaluation and Results **6**
- 6.1 Recording session . . . 7
- 6.2 The noise condition . . . 7

6.3 Can the audio features tell the sounds apart? . . . 7
6.4 Testing the learning step on ManiWAV . . . 9
6.5 How robot noise would affect the audio and policy learning . . . 10
6.6 What the results show . . . 10

7 Conclusion and Outlook **11**

1 Literature Review

This section groups the related work into four topics and then summarises what they have in common and what it means for this project.

1.1 Sound as a sensor for manipulation

Early work used sound to estimate things a camera sees poorly, such as the amount of granular material a robot scoops [Clarke et al., 2018] or the liquid height while pouring [Liang et al., 2019]. Gandhi et al. [2020] recorded 15,000 interactions of 60 objects on a tilting tray with contact microphones and showed that the sound alone identifies most objects and helps predict how they move. Mejia et al. [2024] use a contact microphone on the gripper as a tactile sensor. Because its signal is ordinary audio, an encoder pretrained on large audio-visual datasets can be reused, which is not possible for most tactile sensors.

1.2 Combining sound with vision and touch

Li et al. [2023] fuse a camera, a contact microphone and a GelSight tactile sensor and show that they complement each other: vision gives the global scene, audio reports events immediately, and touch gives local geometry. Du et al. [2022] learn audio-visual policies on a Franka Panda for tasks where the camera cannot see the important object, such as pulling keys out of a bag. Lee et al. [2019] study peg insertion with vision and force sensing and show that for insertion the contact signal carries the information the camera is missing.

1.3 Learning manipulation from demonstrations

Diffusion Policy [Chi et al., 2023] and ACT [Zhao et al., 2023] learn from demonstrations and predict a chunk of future actions instead of a single step. The Universal Manipulation Interface (UMI) [Chi et al., 2024] is a handheld gripper with a camera for collecting such demonstrations without a robot. ManiWAV [Liu et al., 2024] adds a contact microphone to the UMI finger and trains a diffusion policy on images, audio and pose for tasks such as wiping and pouring. It also shows a practical problem: on the robot, motor noise changes the audio compared with the handheld demonstrations. The authors reduce this gap by mixing recorded robot noise and background sounds from ESC-50 [Piczak, 2015] into the training audio.

1.4 Audio representations

The Audio Spectrogram Transformer (AST) [Gong et al., 2021] is the encoder used by ManiWAV and in this project. It cuts a log-mel spectrogram into patches, like an image, and processes them with a Transformer. This project uses weights pretrained on AudioSet [Gemmeke et al., 2017], a large collection of everyday sounds, while ManiWAV trains its AST from scratch. Augmentation such as SpecAugment [Park et al., 2019] makes such models more robust to new recording conditions.

1.5 General points and relation to this project

Four points appear across these works, and each shaped a decision in this project.

P1: Sound helps where the camera cannot see, for example at a contact hidden by the gripper. This is why cleaning and peg insertion were chosen as tasks (Section 5.1).

P2: The robot works use contact microphones. The recordings in this project used an air microphone (Section 4), so a contact microphone is the first hardware step for the robot experiment.

P3: Audio is added to vision and robot state, not used instead of them, and policies use a short history to predict chunks of actions. The pipeline therefore publishes audio features for a policy and prepares action chunks based on the recorded poses (Section 4.4).

P4: Timing and robot noise are the practical problems. Every message in the pipeline carries its capture time (Section 4.3), and robot noise is discussed in Section 6.5.

2 Motivation and Research Questions

The robot in this project already uses vision and its joint state. The goal is to prepare contact audio as an additional input for this robot, because in contact-rich tasks the camera often does not show clearly what happens when the robot touches an object or a surface. This leads to two research questions:

RQ1: Can contact audio provide task-relevant information about contact events, object state, or material when vision and robot proprioception are ambiguous or incomplete?

RQ2: Can contact audio be acquired, represented, and time-aligned in ROS 2 reliably enough to serve as an input to a robot policy alongside vision and robot state?

RQ1 can only be answered on the real robot, by comparing policies with and without audio. RQ2 is the engineering question addressed by the software in this project, which prepares that experiment.

3 Project Scope

The idea of this project is an experiment on the real robot: does a policy that hears its contact sounds perform better than one that only sees? To prepare it, the project covers five parts:

1. acquire live microphone audio, or replay a WAV file through the same interface;
2. transform the waveform into a documented log-mel spectrogram;
3. publish the spectrogram in ROS 2 and encode it into features in a separate ML node;
4. keep the capture timestamps and align the audio windows offline with `/joint_states`, `/tf`, and camera data;
5. assemble the observations a policy consumes and derive the matching actions from the recorded trajectories.

4 From Audio Waveform to a ROS 2 Pipeline

This section describes the software built in this project. It explains how the audio signal is represented, how the processing is split into two ROS 2 [Macenski et al., 2022] nodes, how the robot state is aligned with the audio, and how observations and actions for a policy are produced. Because two different kinds of microphone appear in this report, the section starts with the sensor.

Contact microphone and air microphone. A *contact microphone*, as used in the robot works of Section 1, is a piezoelectric element attached to a solid part such as a gripper finger; it measures the vibrations travelling through that part. The recordings in this project were made with an ordinary *air microphone*, a USB condenser microphone placed close to the contact point, which also picks up the noise in the room. The software works with both, but the signals differ, so results from the air microphone do not transfer directly to a contact microphone.

4.1 The audio representation

The microphone produces a waveform that describes air pressure (air microphone) or structural vibration (contact microphone) over time. An audio encoder needs a more compact view of this signal, so the waveform is converted into a spectrogram that shows how its frequencies change over time. The implemented preprocessing path is shown in Figure 1.

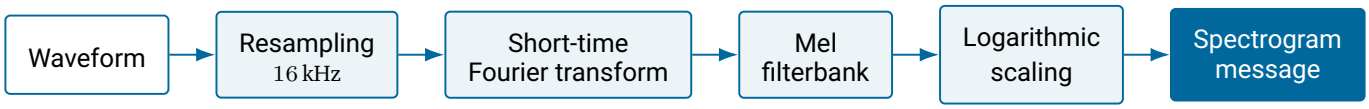


Figure 1: The implemented audio preprocessing chain, from the raw waveform to the spectrogram message published on the ROS 2 topic.

Audio is resampled to $16\,\mathrm{kHz}$. A $25\,\mathrm{ms}$ analysis frame and a $10\,\mathrm{ms}$ frame shift give 98 time frames for a one-second buffer, each with 128 mel-frequency bins. The resulting 98×128 filterbank is published without the padding that AST needs. This keeps the ROS message small and puts model-specific padding and normalisation into the model that requires them.

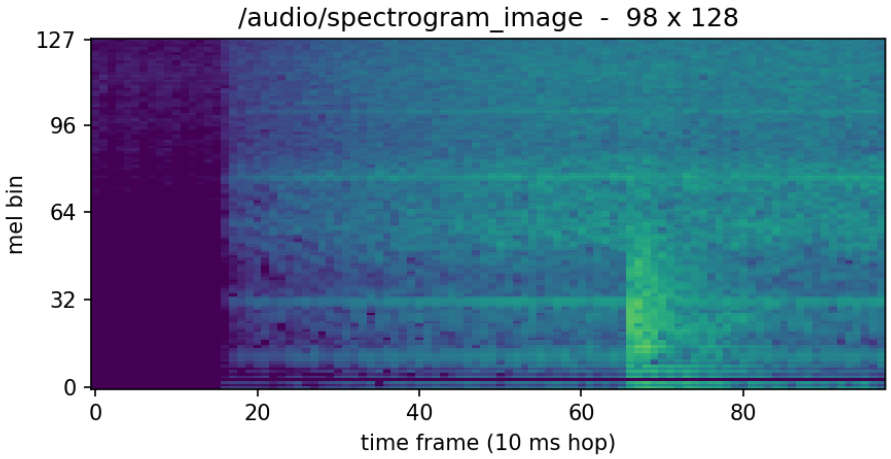


Figure 2: Actual image published by the pipeline for a one-second audio window. The horizontal axis contains 98 time frames and the vertical axis contains 128 mel-frequency bins. This is the same unpadded log-mel representation sent to the ML node; it is not an evaluation result.

4.2 Two nodes

The pipeline consists of two main nodes (Figure 3). The `audio_daq_node` handles everything close to the sensor: capture, buffering, resampling, filterbank computation and publication. It accepts either a live microphone or a WAV file, so the same interface works during development and in the lab.

The `contact_ml_node` subscribes to `/audio/spectrogram` and passes it through an encoder. It publishes the resulting embedding on `/audio/embedding` and the predicted class on `/audio/contact_prediction`. Different encoders can be tried in this node. AST was used as the main encoder, because it is the one used in the literature, but smaller encoders can also be trained from scratch. Two of them are compared with AST in Section 6.3: a small convolutional network (CNN) and simple statistics over the mel bands (`melstats`), which need no neural network. Since the audio processing happens in the other node, changing the encoder does not change the spectrogram.

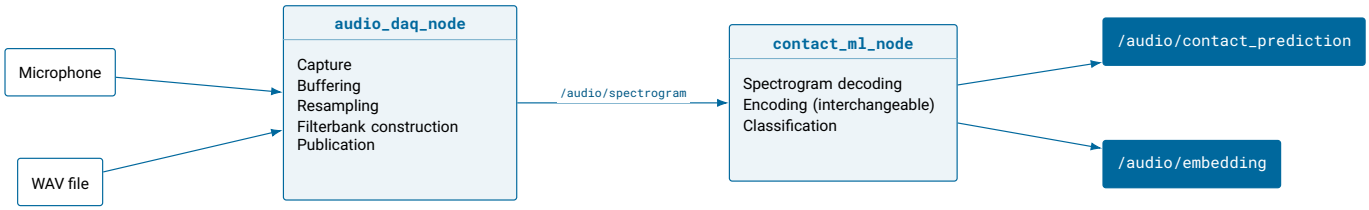


Figure 3: The implemented ROS 2 audio subsystem. Acquisition and representation belong to the DAQ node; encoding and classification belong to the ML node.

4.3 Where robot state enters

The robot's joint positions come from the robot driver, not from the audio pipeline. During an experiment, the audio, the joint positions and the camera images are all recorded into one ROS 2 bag (Figure 4).

After recording, a script looks at each one-second piece of audio and finds where the robot was at the start and at the end of it. Both positions are stored because the robot can move a lot in one second.

To test the script, a simulated robot was used whose joint positions are known exactly at every moment. The script found the correct positions with an error of about 1×10^{-6} rad. When the audio timestamps were shifted on purpose by 100 ms, the error grew to about 0.13 rad, so the test does notice when the timing is wrong. This shows that the script works; the timing on the real robot still has to be tested.

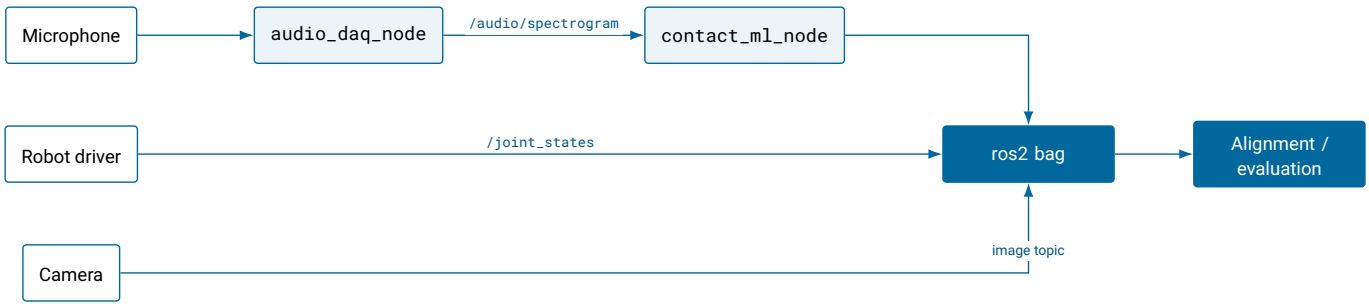


Figure 4: All streams that feed a synchronised recording. The audio path, the robot driver, and the camera are recorded into a single ROS 2 bag, which the offline scripts then align and evaluate.

4.4 Observations and actions for a policy

A classifier says what is happening; a policy decides what to do next. Since the goal is a robot that performs a task, the ML node publishes the full embedding and not only the most likely class. Each embedding also stores when its audio was recorded, so it can later be matched with the robot data.

Another node collects everything the policy needs at one moment: the latest camera image, the audio of the last few seconds, and the current and previous gripper position. How many seconds of audio are used can be set, because ManiWAV found that two to three seconds work better than one.

An offline script turns a recorded bag into pairs of observations and actions. Inspired by ManiWAV, each action has ten values: three for position change, six for rotation change and one for gripper width. A chunk contains 16 future steps. Here, each pose change is measured from one step to the next in the gripper's local frame; the gripper width is the value at the next step. ManiWAV instead expresses future poses relative to the current observation pose, so the targets differ despite having the same size. The script can process hand-guided, teleoperated and handheld pose recordings. It is checked against the simulated robot with the same time-shift test as the alignment.

5 Real-Robot Evaluation Plan

This section describes the planned robot study, what happened in the lab session, and how the next session should be run.

5.1 Choosing a task

The goal is a robot that performs a real task better because it can hear. The task therefore needs sound that carries information the camera cannot get, and that information must change what the robot should do next.

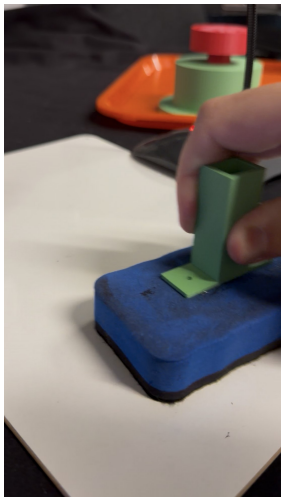
The main proposed task is **cleaning a table**: the robot moves a cloth or sponge over the table to remove a mark. Firm, light and interrupted contact can sound different while looking the same, because the gripper and the cloth cover the contact patch. The hypothesis is that audio helps a policy notice lost contact and correct the stroke.

The second candidate is **peg insertion**, the classic contact-rich benchmark [Lee et al., 2019]. The decisive moment is when the peg slides into the hole and seats. It makes a short, distinct sound, and it happens exactly when the gripper and the part block the camera. Two variants of peg insertion were prepared, which gives three tasks in total (Figure 5):

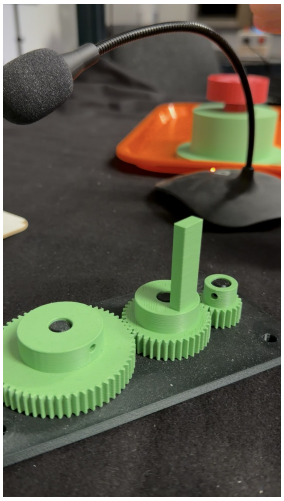
1. **Cleaning the table.** Wiping a surface with a sponge, as described above.
2. **Peg insertion.** A peg is inserted through printed gears of the same shape but different sizes. The contact sound changes with the size of the part and with how firmly and quickly the parts meet.
3. **Peg in hole.** A red peg is pushed into the hole of a green base until it is seated.

5.2 How the robot uses the audio

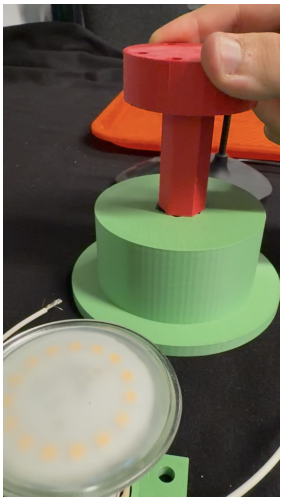
There are two options, and the pipeline supports both. In the first, the audio is reduced to a simple signal, such as "in contact", and a conventional controller reacts to it. In the second, the audio embedding is one input of a learned policy, next to the camera image and the robot state, as in the reviewed works. Everything up to the published spectrogram is the same in both cases, so this decision does not have to be made before recording.



(a) Cleaning the table.



(b) Peg insertion.



(c) Peg in hole.

Figure 5: The intended laboratory tasks. (a) A wiping tool pressed against the working surface. (b) Printed gears of the same shape and material but different sizes, with a peg for insertion. (c) A red peg seated into a green base. The recordings in Section 6 used substitute objects.

5.3 The lab session

The plan was to collect demonstrations by teleoperating the Franka arm [Haddadin et al., 2022] with a SpaceMouse while audio, camera images and robot state were recorded together. With enough demonstrations, two imitation policies would be trained, one with and one without audio, and compared by task success.

This could not be done. The ROS 2 teleoperation packages were installed, but the arm could not be commanded reliably because of a configuration problem on the lab setup, and fixing it did not fit into the time of the session. Contact sounds were therefore recorded by hand using substitute objects and the same acquisition script the pipeline uses for live input. These recordings test the acquisition and representation, but they contain no joint states, no camera images and no trajectories, so no policy can be trained on them. Section 6 reports what they show.

5.4 Plan for the next session

In the next session all streams go into one ROS 2 bag: raw audio, `/audio/spectrogram`, `/audio/contact_prediction`, `/joint_states`, `/tf`, camera images and, for teleoperation, the command topic. Raw audio is kept so that the representation or encoder can be changed later without recording again. One bag does not synchronise clocks by itself: audio and robot state should be captured on the same computer, or the clock offset must be measured. The session has three recording blocks:

1. **Robot self-noise.** The arm performs the task motion in free space without touching anything. This takes a few minutes and gives the noise that every contact sound has to rise above (Section 6.5). If this noise dominates, the microphone mounting or the speed must change before anything else is recorded.

2. **Task demonstrations.** Each episode is one complete, successful execution, with the object position and start pose varied between episodes. Natural corrections, such as regaining lost contact, are kept because they teach the policy to recover.

3. **A second surface or part**, if time allows, to see whether the results hold beyond one setup.

The alignment script is run on every bag before the session ends, because the end of the session is the last point at which a timing problem can still be fixed with the robot available. The most important preparation is hardware: a contact microphone on the gripper (P2 in Section 1.5).

6 Evaluation and Results

Two sources of data were evaluated. The hand recordings with substitute objects (Section 5.3) test acquisition, representation and whether the audio features distinguish three sound categories. Because these recordings contain no robot data, the learning step was tested on a published contact-audio dataset (Section 6.4). The section ends with a discussion of how robot noise would change these results.

6.1 Recording session

The recordings used substitute objects: felt moved over paper for cleaning (`table_cleaning`), placement sounds made with a coin, a metal razor and a ceramic cup for the planned peg-insertion task (`put_sizes`), and a cup placed into a plate for the planned peg-in-hole task (`red_green`). The last two are sound examples for the planned tasks, not tests of actual peg insertion.

All recordings were made by hand with a USB condenser microphone, an air microphone, placed close to the contact point. Bags were written at 48 kHz mono in chunks of 2048 frames. Eighteen takes of 15 seconds were recorded, three per sound category and noise condition, and each was checked right after capture.

The transport layer worked in every take. All eighteen bags decode with no gaps, no overlaps and no clipped samples, and the chunk indices advance by exactly 2048 frames. The audio was reconstructed from the sample index of each chunk, so a dropout would have shown up as a hole instead of being hidden.

Figures 6–8 show one take per sound category with and without noise. Wiping produces broad friction sounds, `put_sizes` contains separate impacts, and `red_green` contains placement sounds. With noise on, the events stay visible, but the background level in the spectrogram rises.

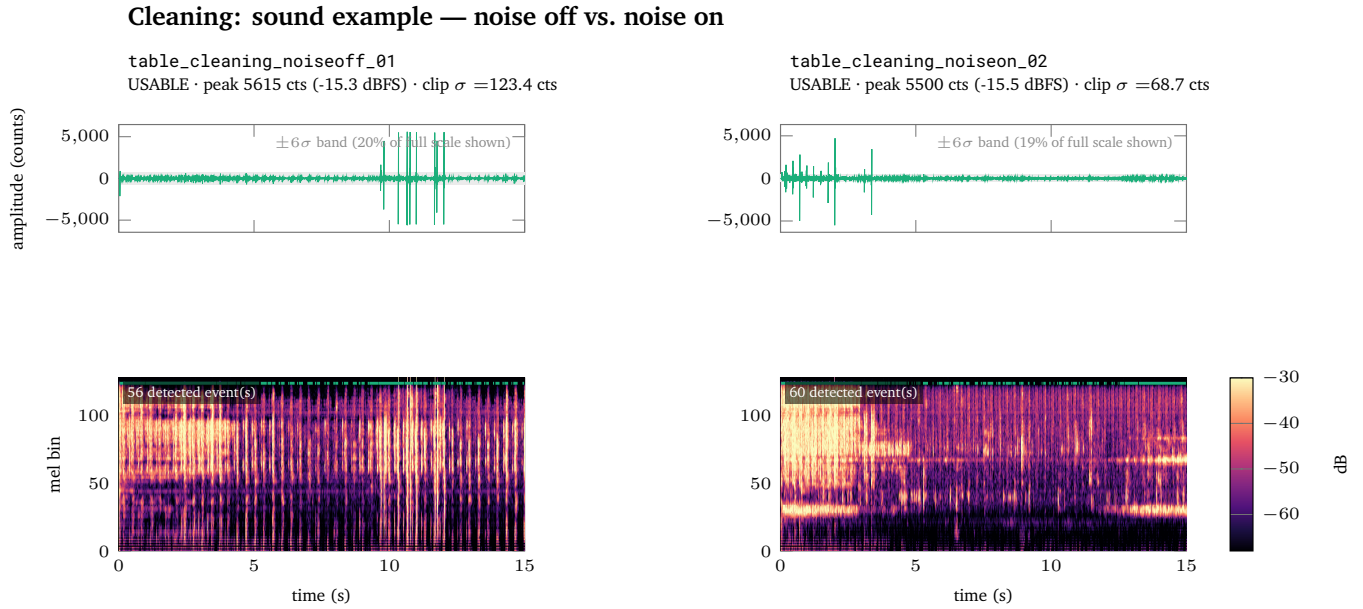


Figure 6: Cleaning sound example: felt on paper, noise off (left) and noise on (right). The shaded band is $\pm 6\sigma$, where σ is the standard deviation of the whole recording, including contact sounds. It is not the noise floor. The spectrogram uses the same filterbank settings as the live pipeline, but is computed over the whole clip without the live path's peak scaling of each one-second window.

6.2 The noise condition

Two reference takes without any contact were recorded, one with the noise source off and one with it on. The noise source raises the RMS level by 22.2 dB, from -69.7 to -47.5 dBFS, and the estimated noise floor by 6.4 dB. This floor is the median RMS level of the quietest 20 % of frames. The mean event-to-floor ratio is lower under added noise in all three sound categories. The event marks in Figures 6–8 show runs of at least three frames more than 6 dB above this floor; they are detected sound events, not labelled contacts.

The added noise, broadband noise played from a phone loudspeaker, is not the same as the noise of the Franka arm. Its spectrum differs, and it reaches the microphone through the air, while motor noise mostly travels through the arm into the tool. This experiment therefore measures robustness to one ambient noise at a known level, not to robot noise. Section 6.5 discusses the difference.

6.3 Can the audio features tell the sounds apart?

Features were computed with the pipeline's own spectrogram code: 1.0 s windows every 0.5 s, each giving a 98×128 filterbank, which was checked by assertion. The three encoder bundles of the ML node produced the embeddings.

The classifier uses cosine nearest-centroid matching: it compares each window's features with the average features of each sound category from the other clips. The category with the most window votes becomes the clip prediction. This is repeated with each clip

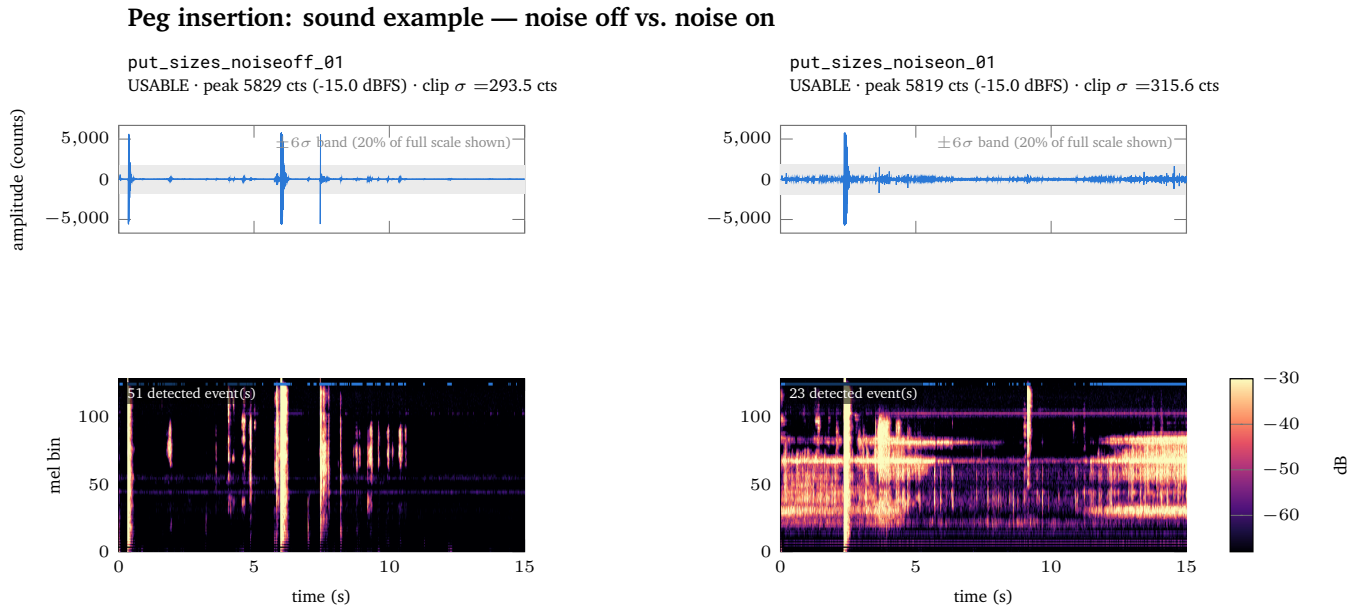


Figure 7: Placement sounds for the planned peg-insertion task (`put_sizes`), noise off (left) and noise on (right). The waveform band and spectrogram follow Figure 6.

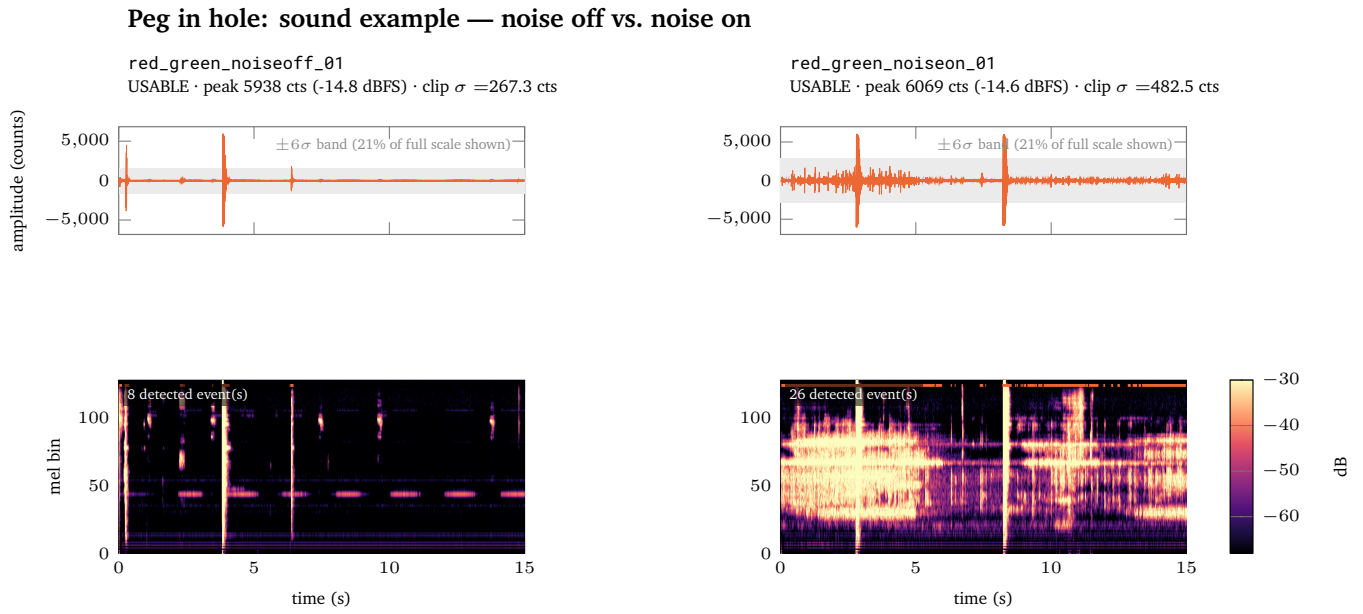


Figure 8: Cup-into-plate sounds for the planned peg-in-hole task (`red_green`), noise off (left) and noise on (right). The waveform band and spectrogram follow Figure 6.

held out once (leave-one-clip-out). The chance level is estimated from 2,000 shuffles of the labels between clips. Clips, not windows, are the unit, because overlapping windows from the same recording are not independent. A random split of windows would put parts of the same recording into training and test data and overestimate the accuracy.

Table 1: Separation of the three sound categories by encoder. Per-clip: correctly classified clips. p: fraction of label shuffles with accuracy at least as high as the measured result, rounded to three decimals.

Encoder	Silhouette	Per-clip	p
`cnn`	0.057	9/18	0.164
`melstats`	0.047	10/18	0.090
`ast`	0.063	**15/18**	**0.001**

The classifier using AST features gets 15 of 18 clips right ($p \approx 0.001$); the CNN and mel-statistics results are not clearly above chance ($p = 0.164$ and $p = 0.090$). The AST-feature classifier gets all `table_cleaning` and `put_sizes` takes right, and all quiet `red_green` takes. Its three errors are the noisy `red_green` takes, all predicted as `put_sizes`. Both categories contain placement sounds, so noise may hide a useful cue. Material and recording differences could also explain the confusion. Figure 9 shows the embeddings.

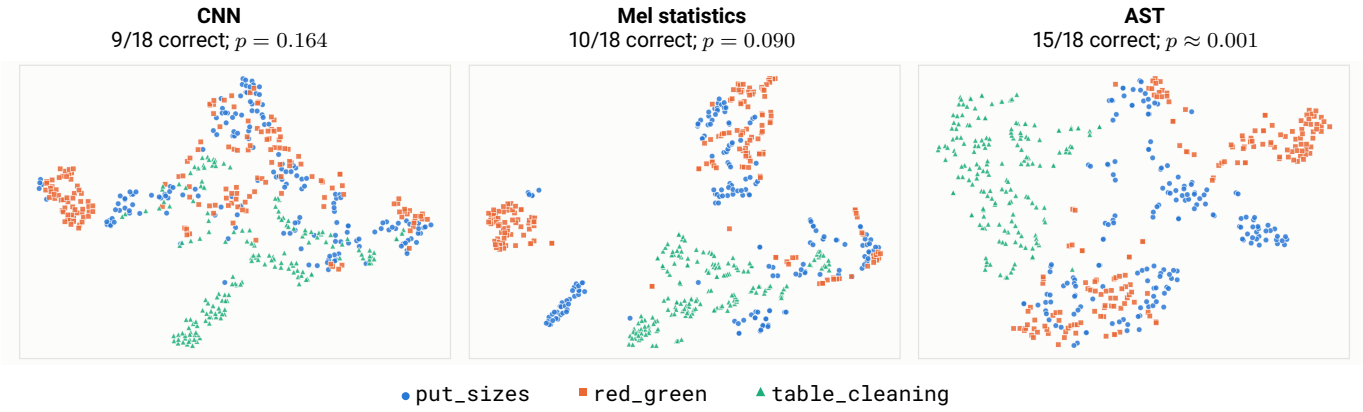


Figure 9: t-SNE [van der Maaten and Hinton, 2008] view of 522 window embeddings from 18 clips. Colour and marker shape show the sound category. The accuracy and p-values come from the clip-level test in Table 1, not from the positions in this plot.

The result has clear limits. It is one dataset of eighteen clips, so it does not rank the encoders in general. The objects also differ in material, so the classifier may recognise materials as well as actions. It has not been tested on real peg insertion.

6.4 Testing the learning step on ManiWAV

Why this experiment. The recordings from this project lack camera images and pose data, so they cannot provide actions for policy training. To test the learning step, the released `wipe` dataset of ManiWAV [Liu et al., 2024] was used. It was recorded with a contact microphone in a handheld UMI gripper and is close to the planned cleaning task. The question is: *can audio features predict the next motion, and do they help when the current pose and gripper width are already known?*

Setup. The pipeline produced 16 565 AST embeddings from 78 episodes at 15 Hz, using the same 98×128 spectrograms as before. The audio input contains 30 overlapping one-second windows. Their endpoints span about two seconds, while the audio itself spans about 2.93 s. The pose input contains only the current position, orientation and gripper width; it has no pose history or speed input.

The targets use the ten-value layout described in Section 4.4: 16 consecutive pose changes and the next gripper widths. At the assumed 60 Hz data rate, these cover about 0.27 s. ManiWAV's released setup instead samples action steps at about 20 Hz and expresses future poses relative to the current observation pose. A small network was trained with current pose and gripper width only, audio only, or both. Eight random splits each held out 16 of the 78 episodes.

The error is mean squared error (MSE) after each action value is centred and scaled to unit standard deviation. These means and scales, for both inputs and targets, were computed from all episodes before splitting. The constant baseline predicts the full-dataset mean action. Each learned model reports its lowest validation error over 40 epochs. This lets validation data affect preprocessing and model selection, so the scores are early checks, not results on an independent test set.

Table 2: Early action-prediction test on ManiWAV `wipe`: MSE on standardised action targets, mean and sample standard deviation over eight splits. Lower is better. Scaling uses all episodes; model scores use the best validation epoch.

Input	Validation MSE	Change from baseline
Full-dataset mean action	1.0179 ± 0.0695	—
Current pose and gripper width	0.7383 ± 0.0525	$-27.5\,\%$
Audio only	0.8088 ± 0.0687	$-20.5\,\%$
Audio, current pose and gripper width	0.7309 ± 0.0616	$-28.2\,\%$

Result. Audio alone gives $20.5\,\%$ lower mean validation MSE than the constant baseline and beats it in all eight splits. It has higher error than the pose-and-gripper input in all eight splits. Adding audio to that input lowers the mean error by about $1\,\%$, with improvement in five of eight splits. The paired error reduction is 0.0074 ± 0.0162 (mean and sample standard deviation). This suggests that audio carries some information about motion, but its added value beside the current pose and gripper width remains unclear. Sound may also reflect wiping speed; this test does not show which cues the network uses.

Limitations. This test uses a frozen encoder, no images, 78 of the 119 released episodes and a small network instead of a diffusion policy. It measures prediction error, not task success, and does not reproduce ManiWAV. A stronger test needs scaling and a mean baseline computed from training episodes only, plus a separate test set. The eight splits share episodes, so their spread is not a significance test. The $60\,\mathrm{Hz}$ data rate and the choice of `mic_0` were inferred from the data during this run and still need checking against the source recording details.

6.5 How robot noise would affect the audio and policy learning

On the robot, the main noise source is the robot itself. The joint motors and gearboxes, the brakes, the gripper and the cooling fans of the controller all produce sound and vibration. This noise affects the sensor readings in three ways.

1. **It raises the noise floor.** As with the loudspeaker (Section 6.2), weak contact sounds such as light wiping or slow sliding can sink below the floor, and in the spectrogram the whole image becomes brighter. With a contact microphone on the gripper this effect is likely much stronger than with an air microphone, because motor vibration travels through the links into the gripper along the same path as the contact signal.

2. **It changes with the motion.** Unlike the constant loudspeaker noise, motor noise appears when the arm moves and disappears when it stops, and its level and frequency depend on joint speed and acceleration. In the spectrogram it appears as bands that follow the trajectory.

3. **It differs between data collection and deployment.** If demonstrations are recorded with a handheld gripper, as in UMI and ManiWAV, there is no motor noise in the training audio, but there is on the robot. ManiWAV names nonlinear motor noise as one of the two main reasons for this gap, next to unexpected sounds from the robot's own interactions [Liu et al., 2024].

For policy learning, each of these has a consequence. Masking removes contact cues, so the audio branch contributes less and the policy relies on vision and pose again. Motion-dependent noise creates a shortcut: a policy may learn the robot's speed from its sound instead of learning about contact, and then fail when the speed changes. A pose history can also provide motion information, but the test in Section 6.4 used only the current pose and does not establish this shortcut. Finally, new sounds during deployment can change the encoder's features and therefore the actions. ManiWAV reduces this gap by recording ten samples of motor noise on the robot and mixing them, together with ESC-50 background sounds, into the training audio; this worked better than filtering the noise at test time [Liu et al., 2024].

For this project, teleoperating the Franka arm means the motor noise is already in the demonstrations, so the gap in point 3 is small, but the shortcut remains. Four measures follow from this: record the self-noise block first (Section 5); mount the contact microphone with damping and away from the motors; add recorded robot noise to the training audio; and check that the audio features of free-space motion differ from those of contact, so that the policy reacts to contact and not only to motion.

6.6 What the results show

The results show that the pipeline works on real audio: eighteen recordings arrive without gaps, the 98×128 representation holds for the project's recordings and for an external dataset, and the alignment logic detects timing errors in simulation. The AST-feature classifier identifies the three sound categories in 15 of 18 clips, with all errors under added noise. In the early test on contact-microphone data, audio features give lower validation error than a constant baseline. RQ2 is partly answered; live recording with a contact microphone and timing against the real robot still need testing. RQ1 remains open: the planned robot study must test whether audio improves task success beyond vision and pose.

The session also improved the recording procedure. Checking each take right after recording caught four bad takes (a cough, two takes in which the noise did not reach the microphone, and one take 27 dB too quiet), which were recorded again on the spot. A level check before each take (tap the tool, require a peak of at least -20 dBFS, reject clipping) should be added to the recording script. A thirty-second reference recording without contact turns the noise level from an assumption into a measurement.

7 Conclusion and Outlook

This project built the audio software needed for a robot to listen to its contacts. A ROS 2 pipeline records audio, turns it into a documented spectrogram, encodes it with a choice of models, and provides time-stamped features. Offline scripts match these features with poses and build action chunks. First results are promising: a classifier using pretrained audio features separates three sound categories recorded with an air microphone and substitute objects. An early test on a published contact-microphone dataset also suggests that the features contain information about motion.

The next steps follow directly from the literature and from these results:

1. mount a piezoelectric contact microphone on the gripper of the Franka arm and measure the robot's self-noise;
2. record teleoperated demonstrations of cleaning the table and of peg insertion, with audio, camera and robot state in one synchronised bag;
3. train a policy, such as Diffusion Policy, with and without audio, using recorded robot noise as augmentation;
4. compare task success on the real robot, focusing on the moments where contact is made, lost or completed.

With the software in place, these steps need mainly robot time and a contact microphone, and they would answer the question this project started with: does a robot manipulate better when it can hear?

References

Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion Policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023. doi: 10.15607/RSS.2023.XIX.026.

Cheng Chi, Zhenjia Xu, Chuer Pan, Eric Cousineau, Benjamin Burchfiel, Siyuan Feng, Russ Tedrake, and Shuran Song. Universal Manipulation Interface: In-the-wild robot teaching without in-the-wild robots. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024. doi: 10.15607/RSS.2024.XX.045.

Samuel Clarke, Travers Rhodes, Christopher G. Atkeson, and Oliver Kroemer. Learning audio feedback for estimating amount and flow of granular material. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 87 of *Proceedings of Machine Learning Research*, pages 529–550, 2018.

Maximilian Du, Olivia Y. Lee, Suraj Nair, and Chelsea Finn. Play it by Ear: Learning skills amidst occlusion through audio-visual imitation learning. In *Proceedings of Robotics: Science and Systems (RSS)*, 2022. doi: 10.15607/RSS.2022.XVIII.009.

Dhiraj Gandhi, Abhinav Gupta, and Lerrel Pinto. Swoosh! Rattle! Thump! – actions that sound. In *Proceedings of Robotics: Science and Systems (RSS)*, 2020. doi: 10.15607/RSS.2020.XVI.002.

Jort F. Gemmeke, Daniel P. W. Ellis, Dylan Freedman, Aren Jansen, Wade Lawrence, R. Channing Moore, Manoj Plakal, and Marvin Ritter. Audio Set: An ontology and human-labeled dataset for audio events. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 776–780, 2017. doi: 10.1109/ICASSP.2017.7952261.

Yuan Gong, Yu-An Chung, and James Glass. AST: Audio spectrogram transformer. In *Proceedings of Interspeech 2021*, pages 571–575, 2021. doi: 10.21437/Interspeech.2021-698.

Sami Haddadin, Sven Parusel, Lars Johannsmeier, Saskia Golz, Simon Gabl, Florian Walch, Mohamadreza Sabaghian, Christoph Jähne, Lukas Hausperger, and Simon Haddadin. The Franka Emika robot: A reference platform for robotics research and education. *IEEE Robotics & Automation Magazine*, 29(2):46–64, 2022. doi: 10.1109/MRA.2021.3138382.

Michelle A. Lee, Yuke Zhu, Krishnan Srinivasan, Parth Shah, Silvio Savarese, Li Fei-Fei, Animesh Garg, and Jeannette Bohg. Making sense of vision and touch: Self-supervised learning of multimodal representations for contact-rich tasks. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8943–8950, 2019. doi: 10.1109/ICRA.2019.8793485.

Hao Li, Yizhi Zhang, Junzhe Zhu, Shaoxiong Wang, Michelle A. Lee, Huazhe Xu, Edward Adelson, Li Fei-Fei, Ruohan Gao, and Jiajun Wu. See, Hear, and Feel: Smart sensory fusion for robotic manipulation. In *Proceedings of the 6th Conference on Robot Learning (CoRL 2022)*, volume 205 of *Proceedings of Machine Learning Research*, pages 1368–1378, 2023.

Hongzhuo Liang, Shuang Li, Xiaojian Ma, Norman Hendrich, Timo Gerkmann, Fuchun Sun, and Jianwei Zhang. Making sense of audio vibration for liquid height estimation in robotic pouring. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5333–5339, 2019. doi: 10.1109/IROS40897.2019.8968303.

Zeyi Liu, Cheng Chi, Eric Cousineau, Naveen Kuppuswamy, Benjamin Burchfiel, and Shuran Song. ManiWAV: Learning robot manipulation from in-the-wild audio-visual data. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2024. URL https://arxiv.org/abs/2406.19464.

Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. doi: 10.1126/scirobotics.abm6074.

Jared Mejia, Victoria Dean, Tess Hellebrekers, and Abhinav Gupta. Hearing Touch: Audio-visual pretraining for contact-rich manipulation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6912–6919, 2024. doi: 10.1109/ICRA57147.2024.10611305.

Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A simple data augmentation method for automatic speech recognition. In *Proceedings of Interspeech 2019*, pages 2613–2617, 2019. doi: 10.21437/Interspeech.2019-2680.

Karol J. Piczak. ESC: Dataset for environmental sound classification. In *Proceedings of the 23rd ACM International Conference on Multimedia*, pages 1015–1018, 2015. doi: 10.1145/2733373.2806390.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023. doi: 10.15607/RSS.2023.XIX.016.